

Linear Algebra Visualizer — Hướng dẫn xây dựng dự án

Phase 1 · Nền tảng — Python · NumPy · Matplotlib · Streamlit

Tài liệu học tập

Contents

Giới thiệu	1
Giai đoạn 1 — Nền tảng: Ma trận là một phép biến đổi	3
Giai đoạn 2 — Animation biến đổi + tương tác linh hoạt hơn	6
Giai đoạn 3 — Eigenvectors và Eigenvalues	8
Giai đoạn 4 — PCA trên dữ liệu 2D thực tế	10
Tổng kết & bước tiếp theo	12

Giới thiệu

Dự án **Linear Algebra Visualizer** là một web app cho phép nhập ma trận và xem trực quan cách ma trận đó biến đổi không gian — bao gồm eigenvectors, eigenvalues, và ứng dụng PCA lên dữ liệu thực.

Mục tiêu học tập của dự án này:

- Hiểu **bản chất hình học** của ma trận (không chỉ là bảng số)
- Thực hành **NumPy** cho các phép toán đại số tuyến tính
- Thực hành **Matplotlib** để vẽ và tạo animation
- Thực hành **Streamlit** để đóng gói thành web app tương tác

Tài liệu này chia thành **4 giai đoạn**. Mỗi giai đoạn là một mốc chạy được độc lập — bạn nên code xong, chạy thử, sửa lỗi (nếu có) trước khi sang giai đoạn tiếp theo. Đừng đọc hết rồi mới code; hãy code song song theo từng phần.

Chuẩn bị môi trường

```
# Tạo thư mục dự án
mkdir linear-algebra-visualizer
cd linear-algebra-visualizer

# Tạo virtual environment (khuyến khích)
python -m venv venv
source venv/bin/activate          # macOS/Linux
venv\Scripts\activate            # Windows

# Cài thư viện cần thiết
pip install streamlit numpy matplotlib
```

Kiểm tra cài đặt thành công:

```
streamlit hello
```

Nếu trình duyệt tự mở ra một trang demo của Streamlit, môi trường đã sẵn sàng.

Giai đoạn 1 — Nền tảng: Ma trận là một phép biến đổi

Khái niệm cốt lõi

Một ma trận 2×2 không chỉ là bảng số — nó là một **hàm biến đổi không gian 2D**. Khi bạn nhân ma trận A với vector v , bạn đang di chuyển điểm v đến một vị trí mới theo quy tắc mà A mô tả.

Ví dụ, ma trận:

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

kéo dài mọi điểm theo trục x gấp 2 lần, giữ nguyên trục y . Ma trận:

$$A = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

xoay mọi điểm 90 độ ngược chiều kim đồng hồ.

Cách hiểu trực quan nhất: hai cột của ma trận chính là *vị trí mới* của hai vector đơn vị $(1, 0)$ và $(0, 1)$ sau khi biến đổi. Toàn bộ không gian còn lại “ăn theo” hai cột đó.

Mục tiêu của Giai đoạn 1

Xây dựng một Streamlit app đơn giản:

- Người dùng nhập 4 số của ma trận 2×2
- App vẽ vector gốc $(1, 0)$ và $(0, 1)$ (màu xám)
- App vẽ vector sau khi biến đổi (màu đỏ)
- Vẽ luôn hình vuông đơn vị trước/sau để thấy rõ sự méo dạng không gian

Code: app.py

```
import streamlit as st
import numpy as np
import matplotlib.pyplot as plt

st.set_page_config(page_title="Linear Algebra Visualizer", layout="wide")
st.title("🔗 Linear Algebra Visualizer")
st.subheader("Giai đoạn 1: Ma trận như một phép biến đổi không gian")

# --- Sidebar: nhập ma trận ---
st.sidebar.header("Nhập ma trận 2x2")
a = st.sidebar.number_input("a11", value=1.0, step=0.5)
b = st.sidebar.number_input("a12", value=0.0, step=0.5)
c = st.sidebar.number_input("a21", value=0.0, step=0.5)
d = st.sidebar.number_input("a22", value=1.0, step=0.5)

A = np.array([[a, b], [c, d]])
st.sidebar.write("Ma trận A:")
st.sidebar.write(A)

# --- Hình vuông đơn vị (4 góc) ---
unit_square = np.array([
    [0, 0], [1, 0], [1, 1], [0, 1], [0, 0]
]).T # shape (2, 5)
```

```

transformed_square = A @ unit_square

# --- Hai vector cơ sở ---
e1 = np.array([1, 0])
e2 = np.array([0, 1])
e1_new = A @ e1
e2_new = A @ e2

# --- Vẽ ---
fig, ax = plt.subplots(figsize=(6, 6))

# Hình vuông gốc (xám, nét đứt)
ax.plot(unit_square[0], unit_square[1], "--", color="gray", label="Khô")

# Hình vuông sau biến đổi (xanh)
ax.plot(transformed_square[0], transformed_square[1], "-", color="royal",
        linewidth=2, label="Sau biến đổi")
ax.fill(transformed_square[0], transformed_square[1], alpha=0.2, color="royal")

# Vector cơ sở gốc (xám)
ax.quiver(0, 0, e1[0], e1[1], angles="xy", scale_units="xy", scale=1,
         color="gray", width=0.008)
ax.quiver(0, 0, e2[0], e2[1], angles="xy", scale_units="xy", scale=1,
         color="gray", width=0.008)

# Vector cơ sở sau biến đổi (đỏ)
ax.quiver(0, 0, e1_new[0], e1_new[1], angles="xy", scale_units="xy", scale=1,
         color="crimson", width=0.012, label="A·e1")
ax.quiver(0, 0, e2_new[0], e2_new[1], angles="xy", scale_units="xy", scale=1,
         color="darkorange", width=0.012, label="A·e2")

lim = max(3, np.abs(transformed_square).max() + 1)
ax.set_xlim(-lim, lim)
ax.set_ylim(-lim, lim)
ax.axhline(0, color="black", linewidth=0.5)
ax.axvline(0, color="black", linewidth=0.5)
ax.set_aspect("equal")
ax.grid(True, alpha=0.3)
ax.legend(loc="upper left")

st.pyplot(fig)

# --- Thông tin số học ---
col1, col2 = st.columns(2)
with col1:
    st.metric("Định thức (det A)", f"{np.linalg.det(A):.3f}")
    st.caption("det A = tỉ lệ thay đổi diện tích. Âm nghĩa là không gi")
with col2:
    st.metric("Hạng ma trận (rank)", np.linalg.matrix_rank(A))
    st.caption("Rank < 2 nghĩa là không gian bị nén thành 1 đường hoặc")

```

Chạy thử

```
streamlit run app.py
```

Hãy thử các ma trận sau và quan sát:

Ma trận	Hiệu ứng kỳ vọng
$\begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$	Kéo dài theo trục x
$\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$	Shear (trượt nghiêng)
$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$	Xoay 90°
$\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$	Nén toàn bộ mặt phẳng thành 1 đường thẳng ($\det = 0$)

Việc cần làm: chạy app, thử từng ma trận trong bảng trên, đối chiếu xem hình vẽ có đúng với dự đoán của bạn không. Nếu gặp lỗi khi chạy, hãy chụp lại traceback đầy đủ để debug ở lần trao đổi tiếp theo.

Giai đoạn 2 — Animation biến đổi + tương tác linh hoạt hơn

Mục tiêu

Thay vì chỉ thấy trạng thái “trước” và “sau”, giai đoạn này thêm animation nội suy (interpolation) để thấy không gian **biến đổi dần dần** — trực quan hơn nhiều cho việc hiểu ma trận.

Ý tưởng kỹ thuật: nội suy tuyến tính ma trận

Ta không thể “animate” trực tiếp một ma trận một cách đơn giản, nhưng có một cách xấp xỉ tốt: nội suy giữa ma trận đơn vị I và ma trận đích A :

$A(t) = (1 - t) * I + t * A$, với t chạy từ 0 đến 1

Ở $t=0$, $A(t) = I$ (không biến đổi gì). Ở $t=1$, $A(t) = A$ (biến đổi đầy đủ). Đây là nội suy tuyến tính đơn giản — không hoàn hảo về mặt toán học cho các phép xoay lớn, nhưng đủ tốt và trực quan cho mục đích minh họa.

Code: thêm vào app.py

Thêm phần này vào cuối file, thay cho việc chỉ vẽ 1 hình tĩnh (bạn có thể giữ cả 2 bằng tab hoặc checkbox):

```
import time

st.subheader("Giai đoạn 2: Animation biến đổi")

if st.button("▶ Chạy animation"):
    placeholder = st.empty()
    n_frames = 30
    I = np.eye(2)

    for frame in range(n_frames + 1):
        t = frame / n_frames
        A_t = (1 - t) * I + t * A

        transformed_t = A_t @ unit_square
        e1_t = A_t @ e1
        e2_t = A_t @ e2

        fig, ax = plt.subplots(figsize=(6, 6))
        ax.plot(unit_square[0], unit_square[1], "--", color="lightgray")
        ax.plot(transformed_t[0], transformed_t[1], "-", color="royalblue")
        ax.fill(transformed_t[0], transformed_t[1], alpha=0.2, color="royalblue")

        ax.quiver(0, 0, e1_t[0], e1_t[1], angles="xy", scale_units="xy",
                  color="crimson", width=0.012)
        ax.quiver(0, 0, e2_t[0], e2_t[1], angles="xy", scale_units="xy",
                  color="darkorange", width=0.012)

        lim = max(3, np.abs(A).max() + 1.5)
        ax.set_xlim(-lim, lim)
        ax.set_ylim(-lim, lim)
        ax.axhline(0, color="black", linewidth=0.5)
        ax.axvline(0, color="black", linewidth=0.5)
        ax.set_aspect("equal")
```

```
ax.grid(True, alpha=0.3)
ax.set_title(f"t = {t:.2f}")

placeholder.pyplot(fig)
plt.close(fig) # QUAN TRỌNG: tránh memory leak khi vẽ nhiều f
time.sleep(0.03)
```

Lưu ý kỹ thuật quan trọng: mỗi lần gọi `plt.subplots()` trong vòng lặp sẽ tạo một figure mới trong bộ nhớ. Nếu không gọi `plt.close(fig)` sau khi vẽ xong, ứng dụng sẽ rò rỉ bộ nhớ (memory leak) và ngày càng chậm sau nhiều lần chạy animation. Đây là lỗi rất phổ biến khi mới học Matplotlib trong vòng lặp.

Nâng cấp thêm (tùy chọn nếu còn thời gian)

- Dùng `st.slider("t", 0.0, 1.0, 0.0)` thay vì animation tự động, để người dùng tự kéo và xem từng bước — dễ debug và học hơn animation tự chạy.
- Thêm preset dropdown: " Xoay 90° ", " Scale x2 ", " Shear ", " Reflection " để không cần nhập tay.

Giai đoạn 3 — Eigenvectors và Eigenvalues

Khái niệm

Với ma trận vuông A , một vector v (khác vector không) là **eigenvector** nếu:

$$A \cdot v = \lambda \cdot v$$

nghĩa là sau khi biến đổi, v chỉ bị **co giãn** theo hệ số λ (eigenvalue) chứ **không đổi hướng**. Đây là những “trục đặc biệt” mà phép biến đổi hoạt động đơn giản nhất — toàn bộ độ phức tạp của ma trận thu gọn về một con số.

Về mặt hình học: nếu bạn vẽ nhiều vector xung quanh gốc tọa độ rồi biến đổi tất cả bằng A , hầu hết chúng sẽ đổi hướng — trừ các vector nằm trên đường thẳng eigenvector.

Code: file mới `pages/eigen.py`

Streamlit hỗ trợ multi-page app tự động nếu bạn tạo thư mục `pages/`. Tạo file `pages/2_Eigenvectors.py`.

```
import streamlit as st
import numpy as np
import matplotlib.pyplot as plt

st.title("Giai đoạn 3: Eigenvectors & Eigenvalues")

st.sidebar.header("Nhập ma trận 2x2")
a = st.sidebar.number_input("a11", value=2.0, step=0.5, key="e_a")
b = st.sidebar.number_input("a12", value=1.0, step=0.5, key="e_b")
c = st.sidebar.number_input("a21", value=1.0, step=0.5, key="e_c")
d = st.sidebar.number_input("a22", value=2.0, step=0.5, key="e_d")

A = np.array([[a, b], [c, d]])

# --- Tính eigenvalues và eigenvectors bằng NumPy ---
eigenvalues, eigenvectors = np.linalg.eig(A)

st.write("### Kết quả")
col1, col2 = st.columns(2)
with col1:
    st.write("**Eigenvalues ( $\lambda$ ):**")
    for i, val in enumerate(eigenvalues):
        st.write(f" $\lambda\{i+1\} = \{val:.3f\}$ ")
with col2:
    st.write("**Eigenvectors (đã chuẩn hóa):**")
    for i in range(eigenvectors.shape[1]):
        v = eigenvectors[:, i]
        st.write(f" $v\{i+1\} = [\{v[0]:.3f\}, \{v[1]:.3f\}]$ ")

# --- Vẽ nhiều vector xung quanh + eigenvector nổi bật ---
fig, ax = plt.subplots(figsize=(7, 7))

# Vẽ 12 vector xung quanh hình tròn để thấy phần lớn bị "xoay lệch"
angles = np.linspace(0, 2 * np.pi, 12, endpoint=False)
for theta in angles:
    v = np.array([np.cos(theta), np.sin(theta)])
    v_new = A @ v
    ax.quiver(0, 0, v[0], v[1], angles="xy", scale_units="xy", scale=1)
```



```

        color="lightgray", width=0.005, alpha=0.6)
ax.quiver(0, 0, v_new[0], v_new[1], angles="xy", scale_units="xy",
        color="steelblue", width=0.005, alpha=0.6)

# Vẽ eigenvector nổi bật (chỉ vẽ nếu eigenvalue là số thực)
colors = ["crimson", "darkorange"]
for i in range(eigenvectors.shape[1]):
    if np.isreal(eigenvalues[i]):
        v = eigenvectors[:, i].real
        lam = eigenvalues[i].real
        # Vẽ đường thẳng qua eigenvector (kéo dài cả 2 chiều)
        ax.plot([-3 * v[0], 3 * v[0]], [-3 * v[1], 3 * v[1]],
            "--", color=colors[i % 2], alpha=0.5)
        ax.quiver(0, 0, v[0], v[1], angles="xy", scale_units="xy", sca
            color=colors[i % 2], width=0.015,
            label=f"eigenvector {i+1} ( $\lambda={lam:.2f}$ )")

lim = 3.5
ax.set_xlim(-lim, lim)
ax.set_ylim(-lim, lim)
ax.axhline(0, color="black", linewidth=0.5)
ax.axvline(0, color="black", linewidth=0.5)
ax.set_aspect("equal")
ax.grid(True, alpha=0.3)
ax.legend(loc="upper right")
ax.set_title("Vector gốc (xám) → sau biến đổi (xanh)\nEigenvector giữ

st.pyplot(fig)

if np.iscomplexobj(eigenvalues):
    st.warning(
        "Ma trận này có eigenvalue phức (complex) — nghĩa là phép biến
        "chủ yếu là xoay, không có trục nào giữ nguyên hướng thực sự.
        "Thử ma trận đối xứng như [[2,1],[1,2]] để thấy eigenvector rõ
    )

```

Việc cần làm: thử ma trận $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ (đối xứng — luôn có eigenvector thực và vuông góc nhau), sau đó thử $\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ (ma trận xoay thuần túy — sẽ không có eigenvector thực, vì xoay thì mọi hướng đều đổi). So sánh cảnh báo app hiển thị.

Giai đoạn 4 — PCA trên dữ liệu 2D thực tế

Vì sao đây là bước quan trọng nhất

PCA (Principal Component Analysis) chính là ứng dụng thực tế của mọi thứ bạn vừa học: nó tìm eigenvector của **ma trận hiệp phương sai** (covariance matrix) của dữ liệu, để xác định hướng nào dữ liệu “trải dài” nhiều nhất. Đây là kỹ thuật giảm chiều dữ liệu (dimensionality reduction) dùng rất nhiều trong Machine Learning — bạn sẽ gặp lại nó ở Phase 2.

Quy trình PCA (không dùng thư viện có sẵn — tự code để hiểu bản chất)

1. Chuẩn hóa dữ liệu về mean = 0 (trừ đi giá trị trung bình mỗi chiều)
2. Tính ma trận hiệp phương sai của dữ liệu đã chuẩn hóa
3. Tính eigenvalues/eigenvectors của ma trận hiệp phương sai
4. Eigenvector có eigenvalue lớn nhất = hướng dữ liệu trải dài nhiều nhất (Principal Component 1)

Code: file mới pages/3_PCA.py

```
import streamlit as st
import numpy as np
import matplotlib.pyplot as plt

st.title("Giai đoạn 4: PCA trên dữ liệu 2D")

st.sidebar.header("Tạo dữ liệu mẫu")
n_points = st.sidebar.slider("Số điểm dữ liệu", 20, 300, 100)
spread_x = st.sidebar.slider("Độ trải theo x", 0.5, 5.0, 3.0)
spread_y = st.sidebar.slider("Độ trải theo y", 0.5, 5.0, 1.0)
rotation_deg = st.sidebar.slider("Xoay dữ liệu (độ)", 0, 180, 30)
seed = st.sidebar.number_input("Random seed", value=42, step=1)

# --- Tạo dữ liệu giả lập: elip xoay ---
rng = np.random.default_rng(int(seed))
raw = rng.normal(0, 1, size=(n_points, 2)) * np.array([spread_x, spread_y])

theta = np.radians(rotation_deg)
rot_matrix = np.array([
    [np.cos(theta), -np.sin(theta)],
    [np.sin(theta),  np.cos(theta)]
])
data = raw @ rot_matrix.T  # xoay toàn bộ dữ liệu

# --- Bước 1: chuẩn hóa mean = 0 ---
mean = data.mean(axis=0)
centered = data - mean

# --- Bước 2: ma trận hiệp phương sai ---
cov_matrix = np.cov(centered.T)  # shape (2, 2)

# --- Bước 3: eigen decomposition ---
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

# --- Bước 4: sắp xếp theo eigenvalue giảm dần ---
```

```

order = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[order]
eigenvectors = eigenvectors[:, order]

pc1 = eigenvectors[:, 0] # hướng trái dài nhất
pc2 = eigenvectors[:, 1] # hướng vuông góc

# --- Tỷ lệ variance được giải thích ---
explained_ratio = eigenvalues / eigenvalues.sum()

st.write("### Kết quả PCA")
col1, col2 = st.columns(2)
with col1:
    st.metric("PC1 giải thích", f"{explained_ratio[0]*100:.1f}% variance")
with col2:
    st.metric("PC2 giải thích", f"{explained_ratio[1]*100:.1f}% variance")

# --- Vẽ ---
fig, ax = plt.subplots(figsize=(7, 7))
ax.scatter(centers[:, 0], centers[:, 1], alpha=0.4, s=20, color="steelblue")

scale1 = 3 * np.sqrt(eigenvalues[0])
scale2 = 3 * np.sqrt(eigenvalues[1])

ax.quiver(0, 0, pc1[0] * scale1, pc1[1] * scale1, angles="xy", scale_units="xy",
          scale=1, color="crimson", width=0.012, label="PC1 (hướng chính)")
ax.quiver(0, 0, pc2[0] * scale2, pc2[1] * scale2, angles="xy", scale_units="xy",
          scale=1, color="darkorange", width=0.012, label="PC2")

lim = max(abs(centers).max() + 1, 4)
ax.set_xlim(-lim, lim)
ax.set_ylim(-lim, lim)
ax.axhline(0, color="black", linewidth=0.5)
ax.axvline(0, color="black", linewidth=0.5)
ax.set_aspect("equal")
ax.grid(True, alpha=0.3)
ax.legend(loc="upper left")
ax.set_title("Dữ liệu (đã căn giữa) + hai trục chính PCA")

st.pyplot(fig)

st.info(
    "Nhận xét: PC1 (đỏ) luôn nằm dọc theo hướng dữ liệu trải dài nhất.  

    Nếu bạn kéo 'Độ trải theo x' lớn hơn nhiều so với 'Độ trải theo y' thì  

    PC1 sẽ giải thích gần 100% variance – nghĩa là bạn có thể bỏ PC2 đi  

    mà gần như không mất thông tin. Đây chính là ý tưởng giảm chiều dữ liệu."
)

```

Việc cần làm: kéo Độ trải theo x lên 5.0 và Độ trải theo y xuống 0.5, quan sát % variance của PC1 tiến gần 100%. Sau đó thử `rotation_deg = 90` và xem PC1/PC2 có hoán đổi vai trò không.

Tổng kết & bước tiếp theo

Cấu trúc thư mục cuối cùng

```
linear-algebra-visualizer/  
├── app.py                # Giai đoạn 1 + 2 (trang chính)  
├── pages/  
│   ├── 2_Eigenvectors.py # Giai đoạn 3  
│   └── 3_PCA.py          # Giai đoạn 4  
├── requirements.txt  
└── venv/
```

File `requirements.txt`:

```
streamlit  
numpy  
matplotlib
```

Deploy lên Streamlit Community Cloud (miễn phí)

1. Push code lên một GitHub repo (không push thư mục `venv/`)
2. Vào **share.streamlit.io**, đăng nhập bằng GitHub
3. Chọn repo, chọn file chính là `app.py`
4. Deploy — Streamlit tự cài `requirements.txt` và chạy app

Những khái niệm bạn vừa thực hành

- Ma trận như phép biến đổi tuyến tính (Giai đoạn 1)
- Nội suy và animation với Matplotlib, tránh memory leak (Giai đoạn 2)
- Eigendecomposition với `np.linalg.eig` (Giai đoạn 3)
- Ma trận hiệp phương sai và PCA tự code từ đầu (Giai đoạn 4)
- Xây dựng multi-page app với Streamlit

Gợi ý mở rộng nếu muốn nâng cấp dự án lên portfolio

- Thêm chức năng upload file CSV để chạy PCA trên dữ liệu thật (ví dụ Iris dataset)
- So sánh PCA tự code với `sklearn.decomposition.PCA` để kiểm chứng kết quả
- Thêm SVD (Singular Value Decomposition) như một giai đoạn 5 — tổng quát hóa eigendecomposition cho ma trận không vuông